

Preparação de Entrevista — Isaac (Arco Educação) · Software Engineer II Backend (Go)

Time de Identidade & Plataforma — guarda dados sensíveis, gerenciamento de acesso, acelera outras squads.

1. STAR Situations — Experiências Reais

Cada situação abaixo é baseada em fatos verificados no histórico de commits e código do Octopus Pay / Ark Streams.

S1 · Migração de engine de dados em produção sem downtime (Pathom 2 → 3)

Pergunta-gatilho: "Fale de uma vez que liderou uma mudança técnica de alto impacto."

Situation	O Octopus Pay usava Pathom 2 como grafo de resolvers para todas as derivações de dados financeiros. Pathom 3 tinha uma API radicalmente diferente e trazia ganhos reais de performance e composabilidade, mas migrar um sistema com 25+ adaptadores e centenas de resolvers em produção era risco alto.
Task	Liderar a migração sem quebrar integrações ativas de clientes reais (Cielo, Adyen, Pagar.me etc) e garantir que o time entendesse os novos padrões antes de adotar.
Action	Fiz a migração incremental: criei wrappers de compatibilidade que permitiam resolvers Pathom 2 e 3 coexistirem na mesma graph. Documentei os novos padrões de <code>pco/defresolver</code> e conduzi sessões de pair programming com o time (commits "com Miguel", "pelo Bueno"). Cada adaptador migrado vinha acompanhado de testes antes e depois para garantir paridade de comportamento.
Result	Migração concluída sem incidentes em produção. O time ganhou padrões mais explícitos de resolvers e o sistema passou a derivar atributos calculados em tempo real com EQL de forma mais eficiente.

S2 · Design de sistema com risco estruturalmente impossível de bypassar (Ark Streams / Risk Guard)

Pergunta-gatilho: "Como você pensa em segurança e integridade em sistemas financeiros?"

Situation	Em sistemas de trading, a maior fonte de risco catastrófico é uma estratégia conseguir enviar ordem ao executor direto, sem passar pelo
------------------	---

	controle de risco. Um bug ou race condition nessa lógica pode resultar em perda total do capital.
Task	Projetar uma arquitetura onde o bypass do risk manager fosse estruturalmente impossível — não apenas uma convenção de código, mas uma impossibilidade física da topologia.
Action	No design do Ark Streams, a estratégia nunca tem referência ao executor. Ela apenas publica um <code>TradeSignal</code> no NATS. O Risk Guard consome esse evento, valida contra limites de exposição e só então emite um <code>RiskToken</code> assinado. O executor só aceita ordens com token válido. Não há caminho de código que permita uma ordem chegar ao executor sem passar pelo guard — a arquitetura de mensageria torna o bypass impossível mesmo com bug na estratégia.
Result	Sistema de trading pessoal rodando com zero incidentes de risco. O mesmo princípio — invariantes por topologia, não por convenção — aplico ao design de qualquer sistema sensível.

S3 · Suíte de testes de integração em sistema com múltiplos gateways externos (Octopus Pay)

Pergunta-gatilho: "Como você garante qualidade em um sistema com muitas integrações externas?"

Situation	O Octopus Pay tinha 25+ integrações com gateways e antifraudes externos. Cada gateway tinha contratos de API diferentes, comportamentos idiossincráticos e respostas que mudavam sem aviso. Testes que dependiam de chamadas reais eram lentos, frágeis e impossíveis de rodar em CI.
Task	Criar uma suíte de testes que cobrisse os contratos de integração de forma determinística, incluindo testes de rota de API completa, sem depender de infraestrutura externa.
Action	Construímos uma camada de testes com <code>clojure.test</code> + Midje (BDD com <code>facts / fact</code>) + <code>matcher-combinators</code> . Para testes de integração de rota, o servidor Pedestal subia em uma porta livre via <code>fixture (use-fixtures :once)</code> , e <code>clj-http.fake</code> interceptava todas as chamadas HTTP externas, retornando fixtures de resposta real capturadas dos gateways. Adotei disciplina de criar teste para cada bug corrigido — o teste primeiro reproduzia o bug, depois o fix o fazia passar.
Result	Suíte de testes cobrindo autenticação, autorização, validação Malli de schema, contratos de rota (400/200), pipelines de enriquecimento (ClearSale), e modelos de domínio (cartão, cliente, merchant). CI executava tudo sem dependências externas.

S4 · Ingestão de dados em escala com extensibilidade por design (Framework Summon / ETL)

Pergunta-gatilho: "Fale de uma solução técnica que você criou do zero e que teve impacto real."

Situation	O Octopus Pay precisava ingerir dados de múltiplos merchants com estruturas de dados completamente diferentes (VTEX, Loja Integrada,
------------------	--

	Sinatra). A cada novo cliente onboardado, o time precisava escrever código customizado de importação, o que não escalava.
Task	Criar um framework de pipelines ETL que fosse extensível por configuração, não por código — adicionar um novo merchant não deveria exigir tocar no core do sistema.
Action	Desenvolvi o framework Summon como um motor de grafos de execução declarativo em Clojure. Cada etapa do pipeline era um nó do grafo com dependências explícitas. Um novo merchant era integrado declarando um grafo de transformações específico para sua estrutura de dados — o core Summon executava o grafo sem saber nada sobre o merchant.
Result	Onboarding de merchants como Redley, Shopinfo e Saucony sem alterações no core. 60% da atividade de commits do Patrick no projeto envolvia criar/ajustar adaptadores — o Summon tornou isso uma tarefa de configuração, não de engenharia pesada.

S5 · Pipeline event-driven com eliminação de divergência backtest/produção (Ark Streams)

Pergunta-gatilho: "Como você lida com consistência de dados em sistemas distribuídos?"

Situation	Em sistemas de trading, o maior problema arquitetural é o "lookahead bias": o backtest usa dados ou lógica diferentes do que roda em produção, então os resultados do backtest são irreais. Isso é um bug de consistência de dados, não de lógica.
Task	Projetar uma arquitetura onde o backtest e a produção fossem garantidamente o mesmo código, com os mesmos dados, sem possibilidade de divergência.
Action	No Ark Streams, o FractalEngine publica <code>AnalyzedEvent</code> s imutáveis para o NATS. A estratégia <code>AlligatorTrend</code> subscreve em <code>market.analyzed.*</code> — ela nunca lê KV diretamente nem sabe se está em backtest ou produção. O backtester injeta os mesmos <code>AnalyzedEvent</code> s via CSV no mesmo subject NATS. O mesmo binário Go, a mesma topologia de mensageria — apenas a fonte dos eventos muda.
Result	Eliminação de divergência por propriedade estrutural. Latência end-to-end em produção medida em $\sim 43\mu s$. Backtester e grid search rodam com os mesmos resultados que o sistema live, garantindo validade dos backtests.

2. Perguntas Comportamentais Frequentes

"Por que quer mudar para Go se você é especialista em Clojure?"

"Clojure me ensinou a pensar em termos de dados imutáveis, composição de funções e sistemas tolerantes a falhas. Quando comecei a construir o Ark Streams em Go, percebi que esses princípios são portáteis — a linguagem muda, o modelo mental não. Go me dá uma plataforma com ecossistema de tooling mais amplo, performance previsível e uma comunidade crescente de sistemas distribuídos. O isaac usa Go + Kafka + PostgreSQL + Redis — exatamente a stack que eu quero consolidar como linguagem principal."

"Como você colabora com o time e dissemina boas práticas?"

"No Octopus Pay, as melhores práticas foram introduzidas de forma incremental, não via decreto. A migração do Pathom 2 para 3, por exemplo, foi feita em pair programming — não mandei um PR gigante e disse 'revisem'. Cada adaptador migrado virou uma sessão de 'olha o que mudou e por quê'. Testes também foram cultivados assim: quando eu fixava um bug, sempre acompanhava com o teste que o reproduzia. Com o tempo isso virou norma no time, não regra."

"Fale de um momento em que você teve que tomar uma decisão arquitetural difícil."

Usar a S2 (Risk Guard) ou S5 (divergência backtest/produção).

"Como você lida com documentação?"

"No Octopus Pay usávamos Swagger/OpenAPI gerado automaticamente via Reitit — a documentação era derivada do código, não mantida separada, então ela nunca ficava desatualizada. No Ark Streams, o README é a fonte de verdade arquitetural: diagrama de topologia NATS, descrição de cada stream, roadmap de fases. Acredito que documentação que precisa ser sincronizada com código manualmente vai sempre divergir — o melhor é fazer a documentação emergir do código."

3. Perguntas Técnicas Esperadas — Isaac Stack

Go

Tópico	O que revisar	Onde você já tem prática
Goroutines e channels	Go concurrency patterns, select, context cancellation	Ark Streams: coletores concorrentes, fan-out para NATS
HTTP com Chi/Fiber	Middleware, routing, request lifecycle	Ark Streams: Chi v5 + Gorilla WebSocket
Interfaces e composição	Interface embedding, mocking em testes	Ark Streams: coletor/strategy interfaces
Error handling idiomático	errors.Is , errors.As , wrapping	Revisar — padrão Go vs Clojure exception
Testes em Go	testing package, table-driven tests, httptest	Revisar — prática real está em Clojure

Estudo prioritário: testes em Go (testing , httptest , mocks com interfaces) e error handling idiomático.

PostgreSQL

Tópico	O que revisar	Situação atual
Transactions e isolation levels	READ COMMITTED vs REPEATABLE READ, deadlocks	Conhece conceito via Datomic (ACID)
Indexes	B-tree, partial indexes, EXPLAIN ANALYZE	Revisar
<code>pgx</code> ou <code>sqlx</code> em Go	Drivers Go para Postgres	Novo — estudar
Migrations	<code>golang-migrate</code> , <code>goose</code>	Novo — estudar

Situação: PostgreSQL está nas habilidades do CV mas a experiência real é com Datomic. Seja honesto: "Conheço PostgreSQL, mas minha experiência de produção pesada é com Datomic — os conceitos ACID, transactions e modelagem relacional eu domino, a API e os drivers Go para Postgres eu estou solidificando."

Redis

Tópico	O que revisar	Situação atual
Data structures	Strings, hashes, sorted sets, streams	Ark Streams usa Redis KV via NATS
TTL e eviction policies	LRU, LFU	Revisar
Pub/sub vs Streams	Diferença e casos de uso	Sabe via NATS JetStream (análogo)
<code>go-redis</code> em Go	Cliente Go	Novo — estudar

Kafka (isaac usa Kafka + Debezium)

Tópico	O que revisar	Situação atual
Topics, partitions, consumer groups	Paralelismo por partição	Forte via NATS JetStream — conceitos idênticos
Offset management	At-least-once vs exactly-once	Conhece via NATS
Debezium e CDC	Change Data Capture, connectors	Novo — estudar. Debezium captura WAL do Postgres e publica no Kafka
<code>confluent-kafka-go</code> ou <code>sarama</code>	Clientes Go	Novo — estudar

Argumento para entrevista: "Trabalhei com NATS JetStream em produção, que tem os mesmos primitivos que Kafka — subjects mapeiam para topics, consumer groups, retenção configurável, replay de mensagens. A API do Kafka é diferente mas o modelo mental é o mesmo."

Kubernetes (stack isaac inclui K8s + Rancher)

Tópico	O que revisar
Pods, Deployments, Services	Básico — provavelmente será perguntado superficialmente
ConfigMaps e Secrets	Análogo ao SSM que você usa na AWS
Liveness/readiness probes	Saúde de serviço
kubectl básico	Comandos do dia a dia

Nível esperado para SE II: não precisar ser expert, mas entender o modelo de deployment e conseguir debugar um pod com `kubectl logs` / `kubectl describe`.

Terraform (isaac usa Terraform, Patrick tem Pulumi)

Diga diretamente: "Minha experiência é com Pulumi (infra-as-code em código, não HCL), mas os conceitos são idênticos — state, plan, apply, recursos declarativos. Consigo aprender Terraform rapidamente."

4. Perguntas sobre o Time de Identidade (contexto da vaga)

O time cuida de: - **Guarda de dados sensíveis** (PII, dados de clientes) - **Gerenciamento de acesso** (autenticação, autorização, permissões) - **Plataforma** — outras squads dependem do que esse time entrega

Perguntas que podem surgir:

"O que você sabe sobre segurança de dados em sistemas de identidade?"

Fale sobre: autenticação via token (você implementou authentication interceptors no Octopus Pay — authentication_test, authorization_test), validação de schema de entrada com Malli (equivalente a não confiar em dados externos), princípio de menor privilégio.

"Como você pensa em APIs de plataforma — aquelas que outras squads vão consumir?"

Fale sobre: Swagger/OpenAPI como contrato explícito (Octopus Pay), Pathom como API de dados que outras partes do sistema consomem via EQL. Estabilidade de contratos, versionamento, backward compatibility.

"O que é data-driven development para você?"

Fale sobre: Ark Streams (backtester com métricas reais, grid search, latência medida), Octopus Pay (pipelines de dados para decisão de roteamento — qual gateway usar baseado em regras).

5. Plano de Estudos — Sem data de pressão, por prioridade

Estudo em blocos independentes. Avance na ordem, mas cada bloco tem valor próprio — se a entrevista chegar no meio do Bloco 2, você já chega com o mais crítico feito.

Bloco 1 — Identidade (prioridade máxima · FC 4.0 + FC 3.0)

Diretamente no escopo do time de identidade do isaac.

JWT completo — FC 4.0 (Autenticação e autorização: Tokens JWT, ACL e RBAC) - [] Estrutura do JWT: Header, Payload, Signature - [] Geração e validação de tokens - [] Expiração e renovação (refresh token) - [] RSA vs HMAC — quando usar cada um - [] Ataques: None Attack, RS256→HS256, Weak HMAC, XSS, CSRF, Replay Attack - [] Modelos de autorização: RBAC, ACL, ABAC — diferenças e casos de uso - [] Princípio do mínimo privilégio

Keycloak — FC 3.0 (Autenticação e Keycloak · 0%) - [] OAuth2 e OpenID Connect — o que são e como funcionam - [] SSO: Single Sign-On na prática - [] Configuração de realms, clients e roles no Keycloak - [] Integração de uma API Go com Keycloak

Bloco 2 — Stack do isaac (gaps identificados)

Kafka — FC 3.0 (36% → terminar) - [] Consumer groups e particionamento - [] Offset commit: automático vs manual - [] Debezium: CDC do PostgreSQL → Kafka (como funciona, formato de evento) - [] Argumento para entrevista: "Padrões idênticos ao NATS JetStream que uso em produção — consumer groups, retenção, replay. API diferente, modelo mental o mesmo."

PostgreSQL + Go — estudo prático - [] `pgx/v5` : queries, transactions, `pgxpool` - [] `golang-migrate` ou `goose` : migrations - [] EXPLAIN ANALYZE: ler plano de execução - [] Índices: B-tree, partial indexes, compostos

Go idiomático - [] Testes: `testing` , table-driven tests, `httptest.NewRecorder` , mocks com interfaces - [] Error handling: `fmt.Errorf` com `%w` , `errors.Is` , `errors.As` - [] Context propagation: `context.WithTimeout` , `context.WithCancel` - [] Go-Fiber vs Chi: diferenças de API (isaac usa Fiber, Ark Streams usa Chi)

Bloco 3 — Complementar

Kubernetes — FC 3.0 (7% → operacional) - [] `kubectl` : get, describe, logs, exec, port-forward - [] Deployment, Service, ConfigMap, Secret - [] Liveness/readiness probes - [] Nível esperado para SE II: operar, não administrar clusters

Terraform — FC 3.0 (0% → uma sessão) - [] Sintaxe HCL básica - [] Plan, apply, state - [] Argumento: "Minha experiência é com Pulumi — mesmos conceitos, sintaxe diferente."

CI/CD — FC 3.0 (0% → curto, vale fechar) - [] GitOps e ArgoCD - [] Pipeline com GitHub Actions (você já tem no Ark Streams — aprofundar)

Revisão contínua

- [] Rerler as STAR situations deste documento em voz alta periodicamente
- [] Para cada tecnologia estudada: preparar uma frase de 2 linhas conectando ao que você já fez

6. O que NÃO esconder (honestidade estratégica)

Ponto	Como apresentar
PostgreSQL sem experiência pesada de produção	"Conceitos ACID, modelagem relacional e transactions domino via Datomic. Drivers e tooling Go-Postgres estou solidificando."
Kafka sem produção	"Padrões idênticos ao NATS JetStream que uso em produção. API diferente, modelo mental o mesmo."
Kubernetes básico	"Uso Docker e Docker Compose extensivamente. K8s estou no nível de operar — não de administrar clusters."
Projeto Go atual é pessoal	"É onde tenho liberdade de tomar decisões arquiteturais sem restrições — por isso é o projeto que mostra mais como eu realmente penso sistemas."

Gerado em 2026-06-03 com base em análise do repositório motor-de-pagamentos e ark-streams.